LogiEdge: Edge AI for Cold-Chain Monitoring on FreightBridge Trucks

AIML ZG535 Machine Learning on Edge (EC-I)

Group 15

Student Details:

BITS ID	Name	Contributions
2025ae05381	Disha Shetty H	100%
2025ae05382	Murali Krishna Sai Garisapati	100%
2025ae05420	Rajan Yadav	100%
2025ae05680	Santosh Kumar Behera	100%

Repository: https://github.com/santosh-2025ae05680/logibridge

Introduction

FreightBridge operates 85 refrigerated trucks carrying vaccines and medicines, and has already lost ₹28 lakh of vaccine to a refrigeration failure that went undetected. This report describes LogiEdge, an edge pipeline that classifies the cargo compartment as Normal, Warning or Critical on the truck itself, from temperature, compressor vibration and door events. It alerts without depending on continuous cellular coverage and can be updated and monitored across the fleet. All sensor data is simulated, so the results show that the pipeline works end to end, not how accurate it would be on real trucks. Sections 1 to 5 cover the deployment context and hardware, the sensor pipeline and MQTT design, model deployment, optimisation, and MLOps monitoring. Every figure quoted comes from the project's own scripts and logs in the repository.

1. FreightBridge Deployment Context

Constraints. Four constraints drive the design. *Latency:* a refrigeration fault raises cargo temperature about 1 °C per minute, so a Critical breach (above 3 °C off setpoint) appears about three minutes after the fault, while the SLA requires an alert within 90 seconds. On the edge, worst-case detection is bounded by the pipeline: a 30-second window evaluated every 10 seconds (about 40 s), plus inference and under a second to log and publish, roughly 41 s with about 49 s of margin. Cellular round trips of [CITE: figure and source] would also fit while connected, so the real problems are tail delays and absent coverage. *Bandwidth:* streaming 1 Hz temperature, 500 Hz three-axis vibration and door events (4 bytes per sample) is about 518.7 MB per truck per day, 518.4 MB of it vibration. At ₹0.10 per MB that is ₹51.87 per truck per day, about ₹1.32 lakh per month for 85 trucks; alerts alone (an assumed 10 a day of

300 bytes) are about 0.003 MB, roughly 170,000 times less. *Connectivity:* seven locations on the Nashik–Aurangabad route lose coverage for 35 to 90 minutes, during which a cloud-only design classifies nothing, the failure mode behind the ₹28 lakh loss. LogiEdge keeps sensing, inference and alert logging on the truck, queues alerts in a persistent local log and forwards them, in order and with original timestamps, when coverage returns. *Privacy:* raw streams never leave the truck; the broker listens only on 127.0.0.1 and only classified alerts travel upstream over TLS, which narrows the attack surface without replacing device hardening.

Hardware selection. Three options were compared on the Constraint Triangle (performance, power, cost) against a hard 10 W budget for the AI node (Table 1). Power is the dominant vertex: it is the only pass/fail limit, latency does not discriminate (every option meets 90 s by a wide margin) and cost only ranks the survivors. Power eliminates the Jetson (15 W, 50% over budget, three times the cost). The STM32H7 is cheapest and lowest-power but cannot run Docker, a Mosquitto broker or Ansible, on which the architecture and container-based updates depend. The Raspberry Pi 5 with AI HAT+ is the balanced choice. The Hailo accelerator is not needed for a model this small, and the selection is analysis only: all measurements ran on a laptop.

Table 1: Hardware options

Option	Hardware	Power	Price per truck	Within 10 W	Pilot (85)	Full (265)
1	Raspberry Pi 5 + AI HAT+ (Hailo-8L)	7.5 W	₹15,000	Yes	₹12.75 lakh	₹39.75 lakh
2	Jetson Orin Nano Super	15 W	₹45,000	No	₹38.25 lakh	₹119.25 lakh
3	STM32H7 custom MCU	0.4 W	₹3,500	Yes	₹2.98 lakh	₹9.28 lakh

[Figure 1: System architecture. Insert scenario_architecture/system_architecture.png]

Architecture. Figure 1 shows sensors feeding a local Mosquitto broker; a containerised service does preprocessing and TFLite inference and writes alerts to a persistent local log; an uplink agent forwards them over the intermittent cellular link (design only, not implemented).

Arithmetic intensity. For the exercise figures (45 MFLOPs and 18 MB per inference; Pi 5 CPU at 16 GFLOP/s and 12 GB/s), arithmetic intensity is 2.50 FLOP/byte against a ridge point of 1.33, so the model is compute-bound: compute time (2.81 ms) exceeds memory time (1.50 ms), a latency bound about 32,000 times inside the SLA. The Roofline therefore favours cutting arithmetic work (INT8 quantisation, structured pruning, where 35% fewer FLOPs gives about 1.83 ms, or a smaller network) over cutting memory traffic. The trained network is far smaller than the exercise figure, at about 800 parameters and 1.5 kFLOPs.

2. Sensor Pipeline and MQTT Design

Simulator. A Python simulator publishes temperature at 1 Hz, N(4.0 °C, 0.3), vibration RMS at 0.5 Hz, N(0.45 g, 0.05), and OPEN/CLOSE door events, selected by `--anomaly {none|temp_drift|vibration|combined}`. Drift adds 0.08 °C per reading (4.8 °C per minute, much steeper than a real failure, so it is a stated limitation) and the vibration fault steps to N(1.2 g, 0.15).

Topic tree and QoS. [Figure 2: MQTT topic tree. Insert the rendered diagram from data_pipeline/mqtt_architecture.md] Topics sit under `logibridge/trucks/{truck_id}/`; the truck ID is in the topic, so sensor payloads carry only a timestamp and a value, while alerts carry it because they leave the truck. Table 2 gives the QoS per topic. Temperature and vibration use QoS 0: the next reading supersedes a lost one within 1 to 2 s, delivery is over loopback, and QoS 1 would add acknowledgements and an in-flight queue that can grow if the broker stalls. Door events use QoS 1, because losing an OPEN or CLOSE corrupts the state history and a duplicate is harmless. Inference results use QoS 0 since a new one replaces them every 10 s. Alerts use QoS 1 so they arrive at least once after an outage; QoS 2 is rejected because its four-message handshake costs extra round trips on a high-latency link, whereas a unique alert ID lets the operations centre drop duplicates. Status topics are QoS 1, retained, with a Last Will. Subscribers to door events and alerts must request QoS 1, because the delivered QoS is the lower of the two.

Table 2: QoS per topic

Topic	Rate	QoS	Retained
sensors/temperature	1 Hz	0	no
sensors/vibration_rms	0.5 Hz	0	no
sensors/door	on event	1	no
inference	every 10 s	0	no
alerts	on Warning or Critical	1	no
status/{client}	on change	1	yes (Last Will)

Preprocessing. Each stream passes a causal 5-sample moving average. Windows of 30 s with a 10 s step give six features: temperature mean, standard deviation and rate of change, and vibration RMS, peak and kurtosis. Features are z-scored with statistics computed once from 10 minutes of clean Normal data (57 windows) and saved as `training_stats.npy`, never recomputed from live data, then clipped to ±10 because drift windows produce z-scores in the hundreds that would stretch the INT8 input range. Vibration arrives at 0.5 Hz, so a window holds 15 samples and kurtosis is noisy; it is also identical in every simulator mode (-0.594), since shifting and scaling Gaussian noise leaves kurtosis unchanged.

Normalisation experiment. Table 3 shows inference with correct statistics and with statistics shifted by ±3σ. In the deployed configuration nothing changed, because the simulated classes are more than 10σ apart and a 3σ shift (0.13 °C of temperature mean, 0.04 g of vibration RMS) cannot cross the boundary; real overlapping faults would be far more sensitive. Without the clip, the FP32 model fell to 40.4%, with every Warning and Critical window wrong, while INT8 stayed at 100% because its input quantiser saturates at the calibrated range and acts as an implicit clip. The clip and statistics are therefore part of the model's input contract and are versioned and shipped with it.

Table 3: Validation accuracy (57 windows) under shifted statistics

Model	Clip	Correct statistics	+3σ	-3σ

Model	Clip	Correct statistics	+3σ	-3σ
FP32 Keras	±10	100.0%	100.0%	100.0%
FP32 Keras	off	40.4%	40.4%	40.4%
INT8 TFLite	±10	100.0%	100.0%	100.0%
INT8 TFLite	off	100.0%	100.0%	100.0%

Data fusion. LogiEdge fuses at feature level: features are extracted from each sensor independently and concatenated into one six-value vector. Data-level fusion would merge streams of different rates (1 Hz and 0.5 Hz), units and noise and need a larger model on a 10 W node. Decision-level fusion would run a model per sensor, but Warning and Critical differ by the combination of temperature and vibration, which per-sensor models cannot see, and it needs extra models and a combination rule. Feature-level fusion keeps the model small, preserves the interaction and lets one set of statistics cover the whole vector.

3. Model Deployment and Pipeline Mapping

Table 4: The ten pipeline stages mapped to LogiEdge [Replace the stage names with the ten stages from your Module 4 slides, keeping the LogiEdge descriptions.]

#	Stage	What happens in LogiEdge
1	Problem definition and constraints	Three classes, a 90 s alert SLA, a 10 W power budget and fully offline operation set the requirements.
2	Data acquisition	A simulator publishes temperature, vibration RMS and door events to the local Mosquitto broker with per-topic QoS.
3	Preprocessing	A 5-sample moving average, 30 s windows with a 10 s step, and z-scoring with frozen training_stats.npy and a clip at ±10.
4	Feature engineering and fusion	Six features (temperature mean, standard deviation, rate; vibration RMS, peak, kurtosis) fused at feature level.
5	Model training	A 6-32-16-3 MLP (803 parameters) trained on 285 windows from three simulator modes, with a time-ordered split and a two-window purge.
6	Evaluation and validation	100% accuracy on 57 held-out windows (gate 88%) and 100% Class 2 recall (gate 95%), plus the 3σ shifted-statistics experiment.
7	Optimisation	INT8 post-training quantisation (M2, 3.38 KB) and 35% neuron pruning with INT8 (M3, 2.94 KB),

#	Stage	What happens in LogiEdge
		compared by benchmark and Pareto analysis.
8	Conversion and packaging	TFLite models inside a python:3.11-slim container with the model as the last layer and a MODEL_PATH override.
9	Deployment and OTA update	A local registry with versioned tags and a seven-task Ansible playbook; a canary rollout is recommended.
10	Monitoring and maintenance	PSI on P(Normal) over the last 100 inferences every 60 s, alerting above 0.25, with a reference tied to the deployed model.

A 6-32-16-3 MLP trained on 285 windows reached 100% validation accuracy against the 88% gate, and the INT8 model is 35% smaller than FP32 and agrees with it on every window. The service closes windows on message timestamps, publishes each result to `logibridge/trucks/{truck_id}/inference` and raises an alert on entering or escalating Warning or Critical, repeated every 60 s and logged locally. The broker runs as a container bound to 127.0.0.1. Image tags are immutable per release, and the playbook ships the model and its drift reference together.

Docker layer-cache result. The image uses python:3.11-slim with dependencies first, code and normalisation statistics next, ENV and CMD, and the model copy last; `MODEL_PATH` selects a model without a rebuild. After replacing only the model file, the rebuild printed CACHED for every step except the final copy of the model, and the push sent one new layer while the other ten reported that the layer already exists. Table 5 gives the bandwidth: the full image is 120.65 MB compressed against a 4.1 KB model layer, a saving of about 29,000 times for 85 trucks. [Replace with the numbers from your recorded demo run.]

Table 5: Bandwidth for 85 trucks

Update method	Per truck	85 trucks	Cost at ₹0.10/MB
Full image pull	120.652 MB	10,255.41 MB	₹1,025.54
Model layer only	0.00414 MB	0.3516 MB	₹0.035

OTA strategy. With the brief's 280 KB model, ₹0.10 per MB and 85 trucks, one update cycle costs about ₹2.38 for full replacement, ₹2.39 for a canary of 10 trucks followed by 75, and ₹2.44 for shadow mode with on-device comparison (₹53.79 if every shadow output is uploaded), about ₹21 a year at a six-week cadence (Table 6). Cost therefore does not decide; risk does. I recommend a canary. It limits a faulty model to 10 of 85 trucks (a missed Critical alert is the ₹28 lakh failure mode) for about 0.5% extra bytes, and rollback to the previous model retained on the truck costs no bytes, which matters during 35 to 90 minute gaps. Promotion requires Class 2 recall above 95% on a replay set and PSI below 0.25 for 48 to 72 hours. Full replacement exposes all 85 trucks at once, and coverage gaps delay the first sign of trouble; it is cheaper than a canary by about ₹0.01 per cycle. Shadow mode has no exposure but measures agreement with the current model, not correctness, may see no rare faults during its week,

uses one to two weeks of a six-week cycle and never exercises the real alert path. The real INT8 model is only 3.4 KB, so an actual update costs about ₹0.03.

Table 6: Cost per update cycle (280 KB model)

Strategy	MB per cycle	₹ per cycle	₹ per year	Trucks exposed
Full replacement	23.80	2.380	20.7	85
Canary (10, then 75)	23.92	2.392	20.8	10
Shadow, on-device comparison	24.39	2.440	21.2	0
Shadow, upload all outputs	537.88	53.788	467.4	0

4. Optimisation and Pareto Analysis

Variants. M1 is the FP32 TFLite baseline (803 parameters, 5.19 KB). M2 is full-integer post-training quantisation calibrated on the 228 training windows (3.38 KB). M3 applies 35% structured pruning with a PolynomialDecay schedule and then the same INT8 quantisation. For an MLP, filter pruning means removing whole neurons: block pruning over each neuron's full weight column zeroed 11 of 32 and 6 of 16 hidden neurons, the output layer was left intact, and the zeroed neurons were then removed physically (their constant outputs folded into the next bias). The result is a 6-21-10-3 network with 400 parameters and 366 multiply-accumulates, equal to the pruned reference to 3e-9.

Benchmark method. Each trial runs 200 timed inferences after 10 excluded warm-ups, single-threaded on pre-quantised validation inputs. The three variants are interleaved and the order rotated each trial, and the reported latency is the median of five trials; an earlier non-interleaved version made whichever variant ran last look twice as slow. Energy is TDP times CPU share times latency: a 28 W processor with 16 logical CPUs and one busy core gives 1.75 W, so energy follows latency by construction and is no independent evidence.

Table 7: Benchmark results

Variant	Mean latency (ms)	p95 latency (ms)	File size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	0.00230	0.00250	5.19	100.00	0.004025
M2 PTQ INT8	0.00242	0.00268	3.38	100.00	0.004240
M3 Pruned + INT8	0.00241	0.00248	2.94	100.00	0.004227

All variants reach 100.00% validation accuracy and 100% Class 2 recall, above the 95% requirement. Latency is about 2.3 to 2.4 µs for all three; M1 is about 5% faster, which is within trial-to-trial noise (its trial means ranged from 2.28 to 3.23 µs). INT8 and pruning give no speed-up here, because per-call overhead dominates a model this small even though M3 does half the arithmetic. The real differences are file size (M3 is 43% smaller than M1 and 13% smaller than M2) and working memory (545, 1,016 and 3,452 bytes). [Figure 3: Pareto chart. Insert optimisation/results/pareto_chart.png] Treating latency and

energy within 10% as ties, M3 is the only Pareto-optimal variant: it matches the others on accuracy, latency and energy and is the smallest.

Recommendation: M3. The SLA decomposes into about 40 s of worst-case windowing and under 1 s to publish, leaving about 49 s of margin. Inference runs once per 10 s step, so the binding per-inference budget is 10 s. At 2.4 µs measured, and about 0.12 ms even after a conservative 50 times allowance for a Pi 5 core (an assumption to confirm on the device), the margin is roughly 80,000 times. Memory is ample: working tensors of 545 bytes fit in an L1 data cache [verify cache size] and the container used 61.5 MiB of the 8 GB. M3 is chosen on size and memory headroom, not speed. Its risks are reduced capacity (the simulated classes are separable by more than 10σ, so 100% is no evidence of robustness on real, overlapping faults) and an INT8 range calibrated on simulated data. They are mitigated by the canary rollout and promotion gate of Section 3, with M2 retained on the truck as the instant rollback model.

5. MLOps Monitoring and Reflection

PSI monitoring. The monitor compares a reference distribution (300 clean Normal windows through the deployed model, in four bins) with the last 100 inferences every 60 seconds, and prints [LOGIBRIDGE DRIFT ALERT] above 0.25. The confidence score is the probability of the Normal class: the maximum softmax is about 1.0 for every class on this separable data (0.995, 1.000, 0.999), so PSI on it stayed flat at 0.011 through the whole fault in simulation, whereas P(Normal) falls from 0.995 to 0.000. [Confirm this interpretation with the instructor.] Injecting `--anomaly combined` mid-run gave the values in Table 8: the first alert came at the first evaluation after injection (within 5 simulated minutes, about 27 s of real time at 10 times acceleration), PSI peaked at 5.648, and it returned to 0.000 about 1,170 simulated seconds after the anomaly ended, once the 100-inference window had flushed. Values above 1 reflect the 1e-4 floor on empty reference bins, so they mean far above 0.25, not a precise effect size.

Table 8: PSI before, during and after injection (live run, 30 s rehearsal interval)

Phase	Time	PSI	Status
Before	06:59:52	0.000	ok
During	07:00:22	1.903	ALERT
During	07:00:52	4.382	ALERT
During	07:01:22	5.648	ALERT
During	07:01:52	4.717	ALERT
During	07:02:22	1.655	ALERT
After	07:02:52	0.000	ok

Drift type. The injected change is covariate shift: the input distribution moves (a sudden vibration step and an incremental temperature ramp) while the model's mapping is unchanged, and it is detected through the output-score distribution. PSI cannot tell a genuine fault from sensor degradation, so an alert should trigger review, not automatic retraining. [Add your own justification.]

Ansible idempotency. The seven-task playbook deployed the M3 release with ok=7 changed=4, and an immediate re-run reported ok=6 changed=0 skipped=1. The stop task runs only when the model or reference changed, the pull happens only when the tag is missing locally, the start task recreates the container only when its image or settings differ, and the verification is read-only.

Technical difficulty. Pruning with tensorflow-model-optimization 0.8.0 failed with an error that an input tensor must be rank 2. The cause was in the library: its pooling helper ends with a bare squeeze, so a block spanning a whole weight column collapses from shape (1, N) to rank 1 and the next function rejects it. Wrapping that helper to keep the result rank 2 fixed it, and exactly 11 of 32 and 6 of 16 neurons were zeroed. [Replace with your own account of what you tried and how you found the cause.]

Architectural change. The drift monitor kept printing PSI 0.000 as healthy for hours after the simulator stopped, because it reads its last buffer and never notices that no inferences are arriving. I would add a staleness check that alerts when no inference arrives for two evaluation intervals, and have the service publish a heartbeat on its status topic, so that a dead service cannot look healthy.

Conclusion

LogiEdge demonstrates, on simulated data, an edge pipeline that classifies the cargo compartment on the truck, keeps working through coverage gaps, can be updated by shipping a few kilobytes and is monitored for drift. The measured results are consistent: 100% validation accuracy and Class 2 recall, a 2.94 KB model, PSI that rises from 0.000 to 5.648 under fault injection and returns to 0.000, and an idempotent deployment whose second run changes nothing. The limits are equally clear: the simulated classes are trivially separable, the drift is far steeper than a real failure, the hardware choice and energy are analysis and estimation rather than measurements on a Raspberry Pi 5, the uplink agent is not implemented, and the monitor cannot detect missing data. The next steps are to measure on real hardware with real fault data, add a staleness check and a retained rollback model, and implement the uplink.